UNIVERSITY OF ENGINEERING & TECHNOLOGY MARDAN

COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

SUBMITTED BY

NAME : RIFAQ AJMAL

REG NO : 23MDBCS 435

SECTION : CS-A

SEMESTER : 6th

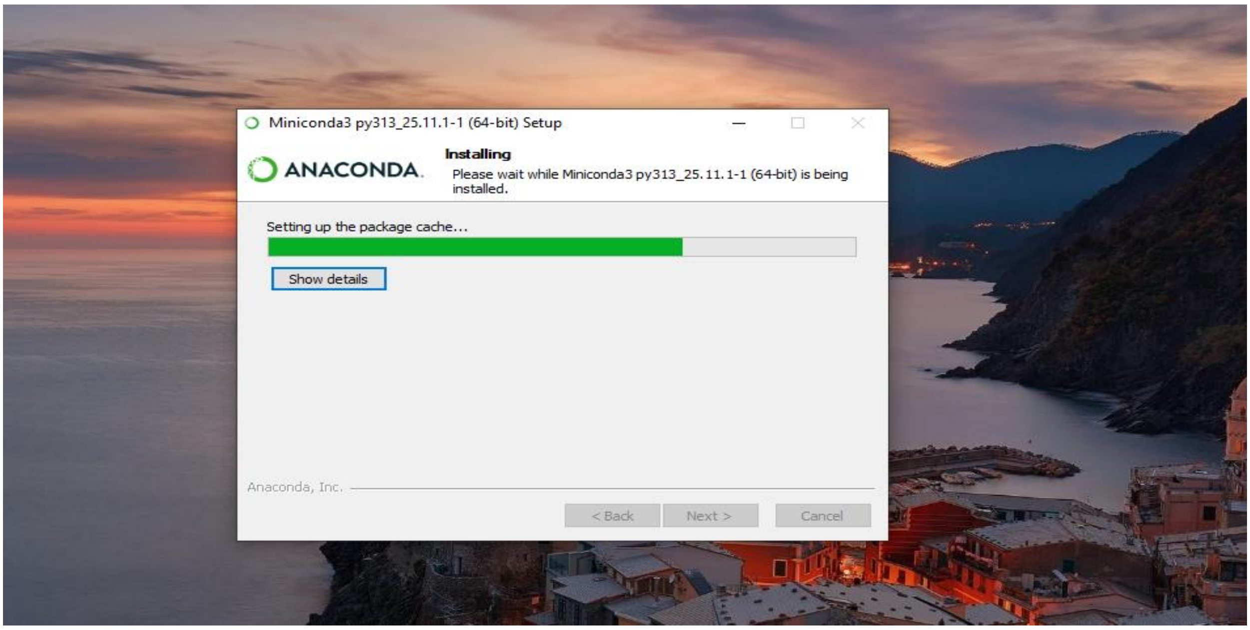
Date: 1/30/2026

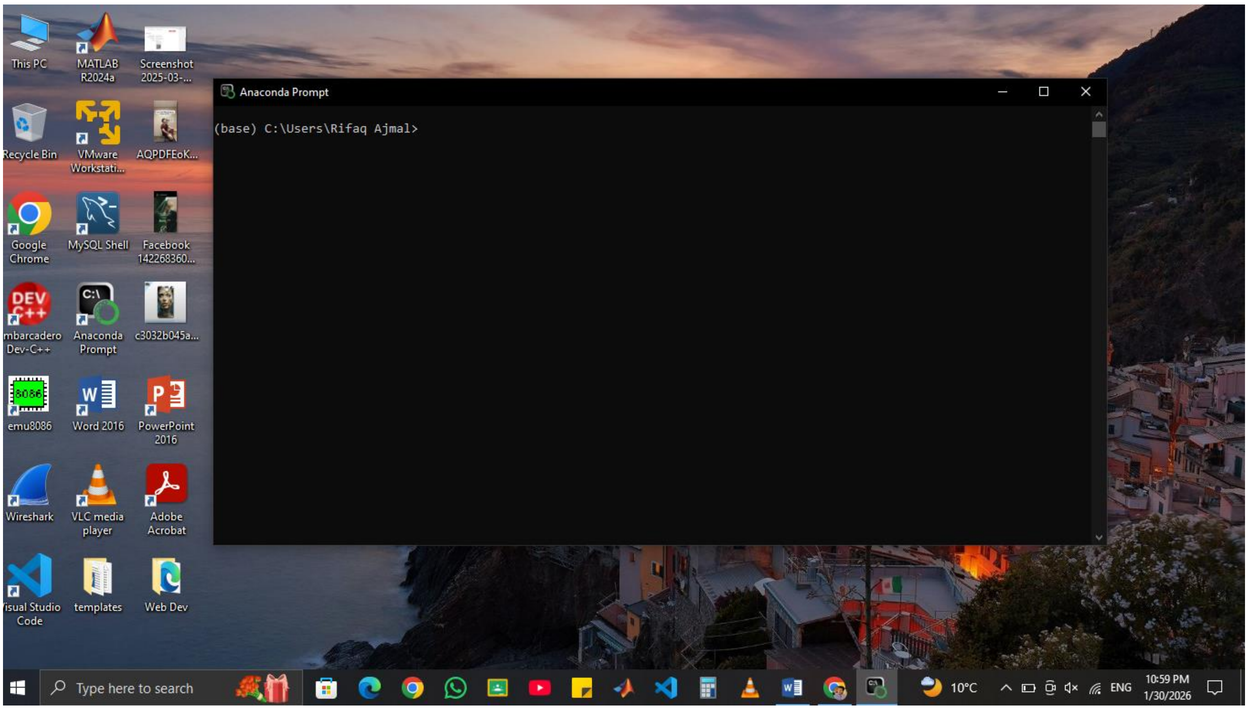
Lab 01: Introduction & MPI Setup

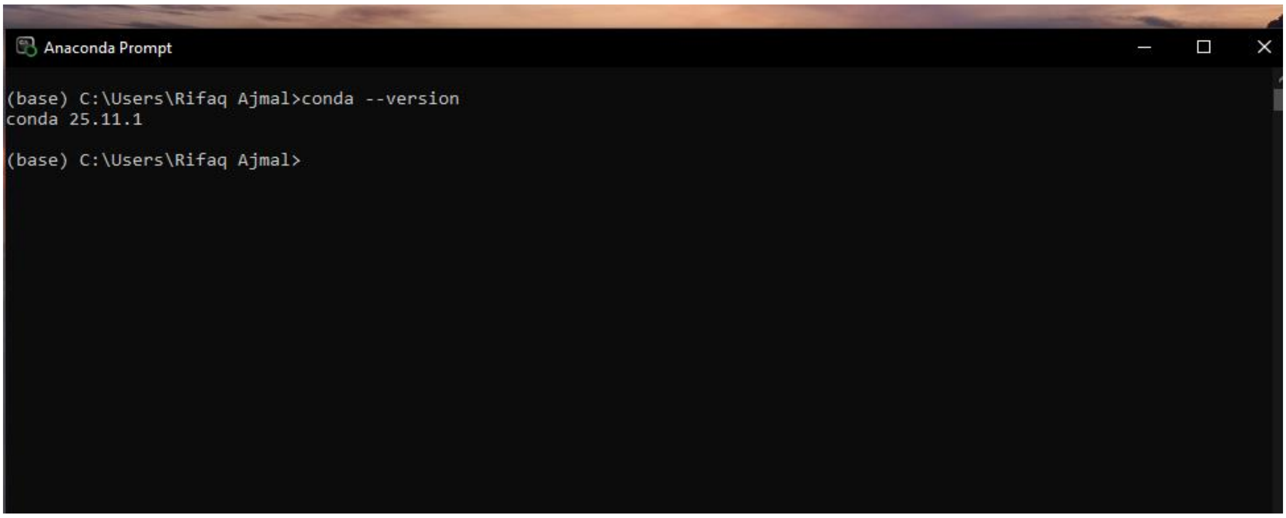
SECTION 1: SCREENSHOTS

========================

1.1 Conda Installation Proof

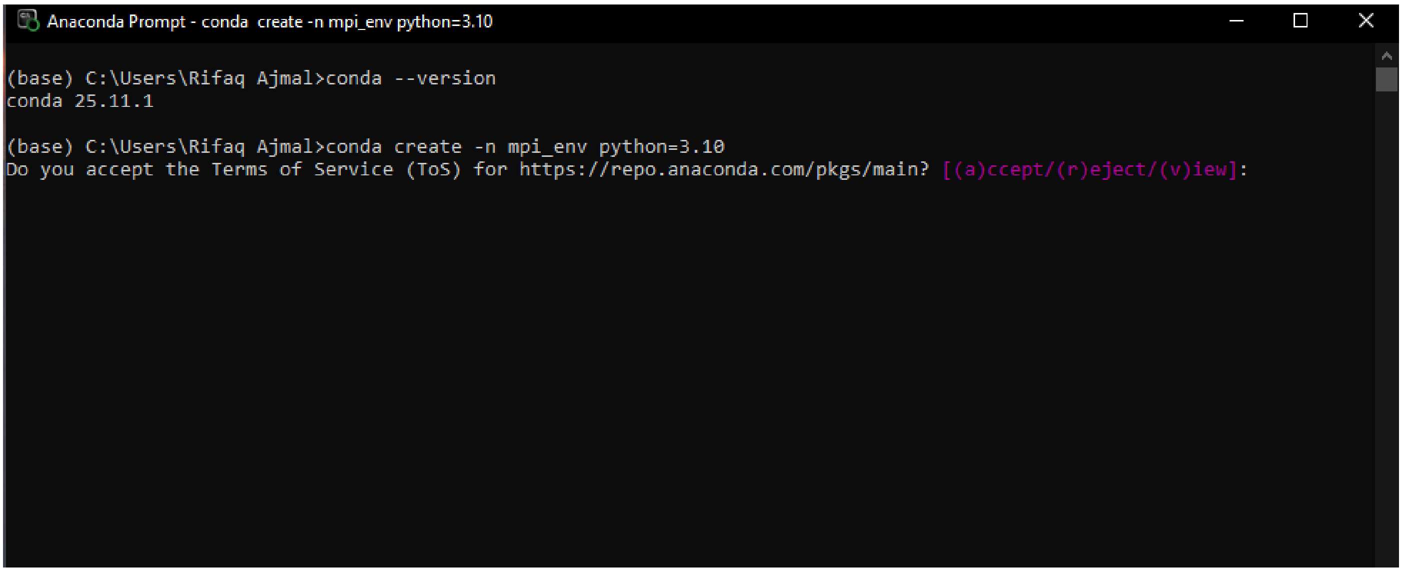




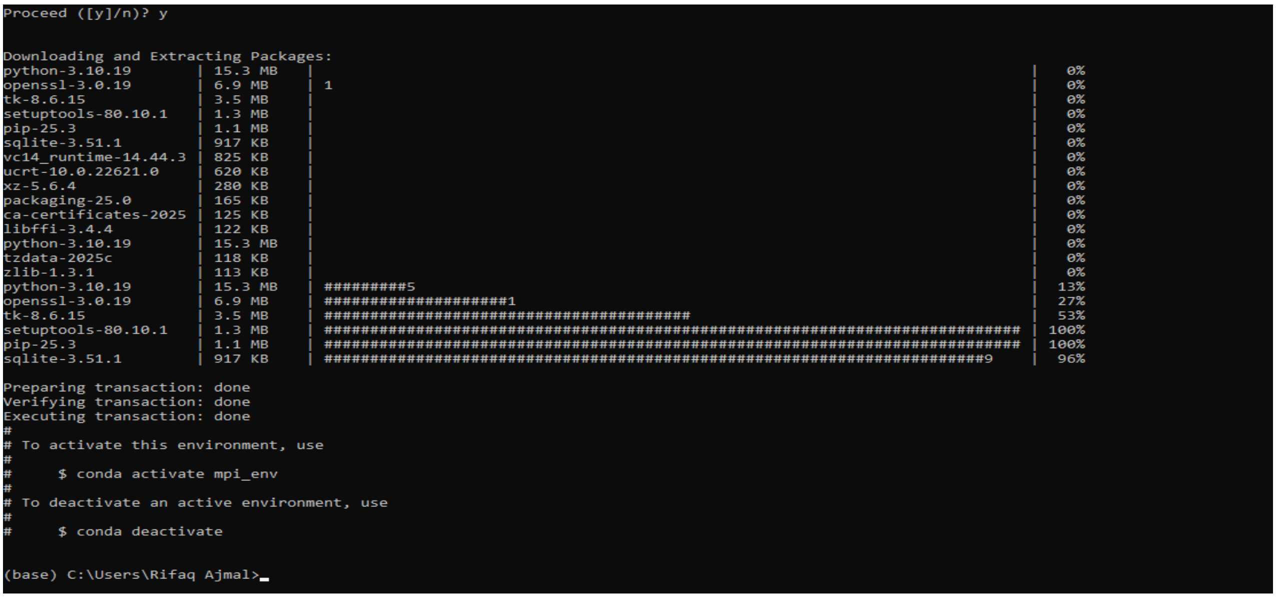


1.2 Environment Activation

CREATE MPI ENVIRONMENT



MPI ENVIRONMENT CREATED



1.3 MPI Installation Proof

ACTIVATE ENVIRONMENT

Activated (mpi_env)

```
Anaconda Prompt

(base) C:\Users\Rifaq Ajmal>conda activate mpi_env

(mpi_env) C:\Users\Rifaq Ajmal>
```

INSTALL MPI (mpi4py)

```
Anaconda Prompt
(mpi_env) C:\Users\Rifaq Ajmal>conda install -c conda-forge mpi4py
```

MPI INSTALLED

```
Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done

(mpi_env) C:\Users\Rifaq Ajmal>
```

1.4 First MPI Output

STEP 4: VERIFY MPI INSTALLATION

- **I wrote**

  Python

- **And type code**

  from mpi4py import MPI

  print("MPI is working")

- **And Result is Shown**

  MPI is working

- **And exit python by**

  exit()

```
(mpi_env) C:\Users\Rifaq Ajmal>python
Python 3.10.19 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 16:41:31) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from mpi4py import MPI
>>> print("MPI is working")
MPI is working
>>> exit()

(mpi_env) C:\Users\Rifaq Ajmal>
```

First MPI Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>mpiexec -n 4 python test_mpi.py
Hello from process 2 out of 4
Hello from process 1 out of 4
Hello from process 3 out of 4
Hello from process 0 out of 4

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>
```

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>mpiexec -n 1 python test_mpi.py
Hello from process 0 out of 1

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>
```

Miniconda installation

✓ MPI environment creation
✓ mpi4py installation
✓ MPI verification
✓ MPI program execution (`mpiexec -n 4`)

SECTION 2: CODE

================

2.1 Basic MPI Program (test_mpi.py)

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

print(f"Hello from process {rank} out of {size}")
```

2.2 Modified MPI Program (modified_mpi.py)

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

print(f"Process {rank} is running among {size} total processes")
```

SECTION 3: LAB REPORT

=======================

3.1 Installation Challenges

Initially, understanding the MPI setup was challenging.
However, using Anaconda made the installation process easier.

3.2 Observations

MPI successfully runs the same program on multiple processes.
Each process has a unique rank.

3.3 Concept Understanding

MPI allows parallel execution of programs using multiple processes.
It is used to improve performance in distributed systems.

3.4 Task Output

Output with -n 4:

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>mpiexec -n 4 python test_mpi.py
Hello from process 2 out of 4
Hello from process 1 out of 4
Hello from process 3 out of 4
Hello from process 0 out of 4

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>
```

Output with -n 1:

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>mpiexec -n 1 python test_mpi.py
Hello from process 0 out of 1

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_1>
```